

PROJECT SUBMISSION
AENEXZ TECH

DIGITAL LOGIC & HDL DESIGN

8-Bit Ripple Carry Adder

Verilog Design & Simulation

Complete Architecture, Logic Verification, and Output Waveform Analysis on EDA
Playground

DONE BY K.ESHWAR
PROGRAMME :- VLSI
BATCH :- JUNE@26

Project Overview



Core Objective

To design an 8-bit binary addition circuit using modular Verilog HDL code and verify its arithmetic functionality through simulation.



Modular Approach

Built structurally by cascading eight 1-bit Full Adders, where the Carry-out (C_{out}) ripples sequentially from one stage to the next.

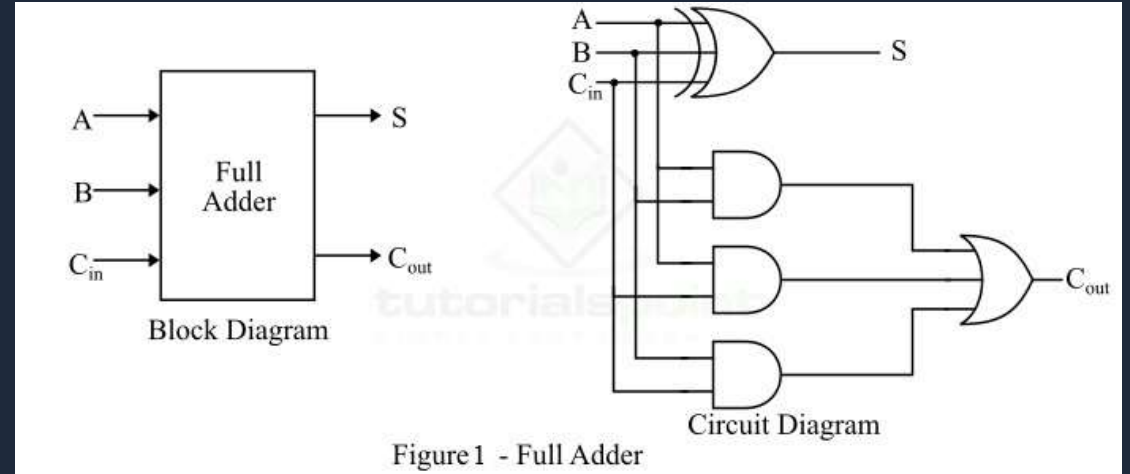


Verification Tool

Simulated using Icarus Verilog on EDA Playground, with signal waveforms captured and analyzed using the EPWave viewer.

Building Block: 1-Bit Full Adder

- ✓ **Inputs:** Takes two 1-bit operands (A, B) and a Carry-in (C_{in}) from the previous stage.
- ✓ **Outputs:** Generates a 1-bit Sum and a Carry-out (C_{out}).
- ✓ **Sum Logic:** Calculated using an odd-parity XOR gate logic across all inputs.
- ✓ **Carry Logic:** Evaluates to high if any two or more inputs are high (1).



Boolean Logic Equations:

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \& B) \mid (B \& C_{in}) \mid (A \& C_{in})$$

8-Bit Architecture & Carry Ripple

-  **Sequential Chaining:** Stage 0 (FA_0) receives the external carry input $C_{\{in\}}$. Its output $C_{\{out0\}}$ drives $C_{\{in1\}}$ of FA_1.
- **Ripple Effect:** The carry signal "ripples" sequentially through all 8 full adders (FA_0 → FA_7) until reaching final $C_{\{out\}}$.
-  **Bus Widths:** Operands $A[7:0]$ and $B[7:0]$ produce an 8-bit output $Sum[7:0]$ plus a single bit Carry-out ($C_{\{out\}}$).

8

Full Adder Units

8-Bit

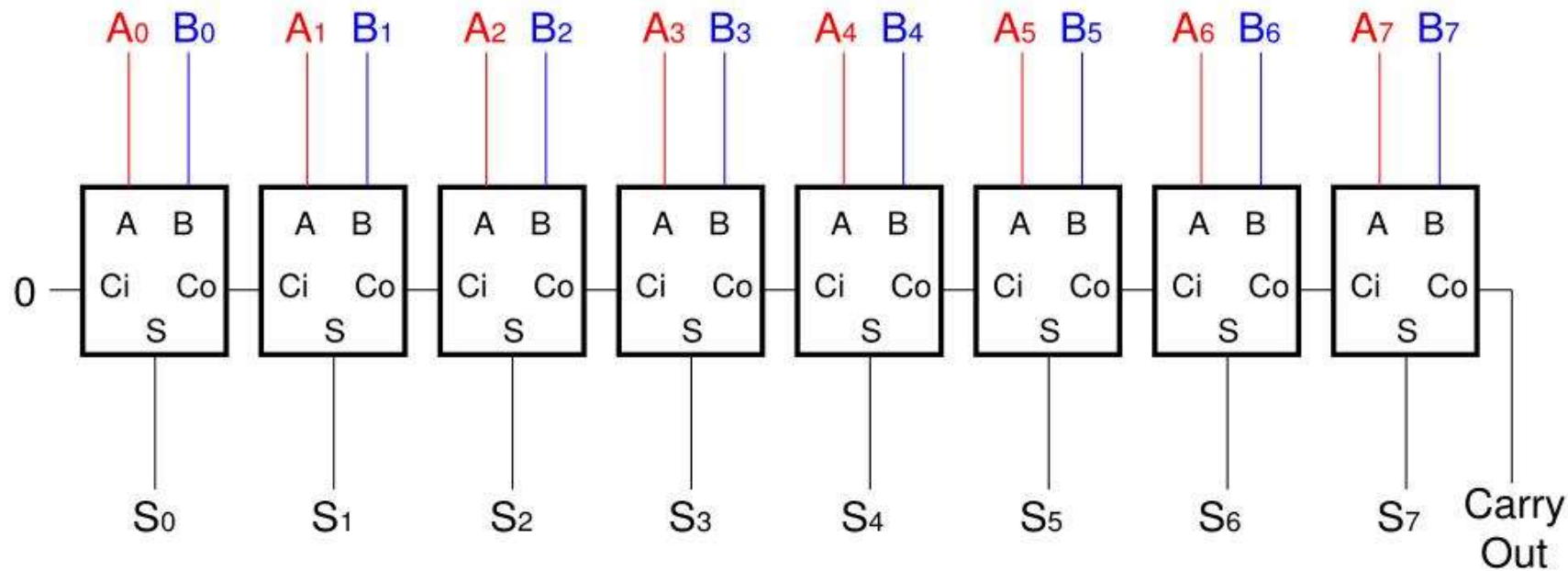
Data Bus Width

0 to 255

Total Output Range

Full Adder Application: 8-Bit Ripple-Carry Adder

- Constructed by connecting 8 full adders together



VERILOG DESIGN CODE FOR 8 BIT RIPPLE CARRY ADDER

```
// 1-bit Full Adder module
module full_adder (
    input wire a,
    input wire b,
    input wire cin,
    output wire sum,
    output wire cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule

// 8-bit Ripple Carry Adder module
module ripple_carry_adder_8bit (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire cin,
    output wire [7:0] sum,
    output wire cout
);
    wire [6:0] c; // Internal carry signals

    // Instantiating 8 full adders in a ripple chain
    full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c[0]));
    full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[0]), .sum(sum[1]), .cout(c[1]));
    full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[1]), .sum(sum[2]), .cout(c[2]));
    full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[2]), .sum(sum[3]), .cout(c[3]));
    full_adder fa4 (.a(a[4]), .b(b[4]), .cin(c[3]), .sum(sum[4]), .cout(c[4]));
    full_adder fa5 (.a(a[5]), .b(b[5]), .cin(c[4]), .sum(sum[5]), .cout(c[5]));
    full_adder fa6 (.a(a[6]), .b(b[6]), .cin(c[5]), .sum(sum[6]), .cout(c[6]));
    full_adder fa7 (.a(a[7]), .b(b[7]), .cin(c[6]), .sum(sum[7]), .cout(cout));

endmodule
```

VERILOG TESTBENCH CODE FOR 8 BIT RIPPLE CARRY ADDER

```
`timescale 1ns / 1ps

module tb_ripple_carry_adder_8bit;

    // Testbench signals
    reg [7:0] a;
    reg [7:0] b;
    reg cin;
    wire [7:0] sum;
    wire cout;

    // Instantiate the Device Under Test (DUT)
    ripple_carry_adder_8bit dut (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

    initial begin
        // Waveform generation for EDA Playground (EPWave)
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_ripple_carry_adder_8bit);

        // Monitor outputs in the console
        $monitor("Time=%0t | A=%d (0x%0h) | B=%d (0x%0h) | Cin=%b | Sum=%d (0x%0h) | Cout=%b",
            $time, a, a, b, b, cin, sum, sum, cout);

        // Test Case 1: Simple addition without carry
        a = 8'd15; b = 8'd20; cin = 1'b0; #10;
```

```
// Test Case 2: Addition with Cin = 1
```

```
a = 8'd45; b = 8'd55; cin = 1'b1; #10;
```

```
// Test Case 3: Maximum 8-bit sum (no carry out)
```

```
a = 8'd128; b = 8'd127; cin = 1'b0; #10;
```

```
// Test Case 4: Overflow generation (Cout = 1)
```

```
a = 8'd255; b = 8'd1; cin = 1'b0; #10;
```

```
// Test Case 5: All 1s with Cin = 1
```

```
a = 8'd127; b = 8'd127; cin = 1'b1; #10;
```

```
// Test Case 6: Random stimulus test
```

```
repeat (3) begin
```

```
    a = $random;
```

```
    b = $random;
```

```
    cin = $random % 2;
```

```
    #10;
```

```
end
```

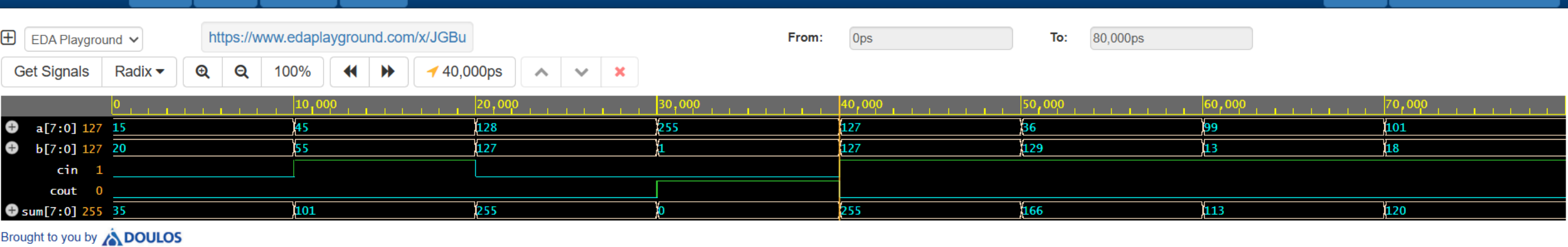
```
$finish;
```

```
end
```

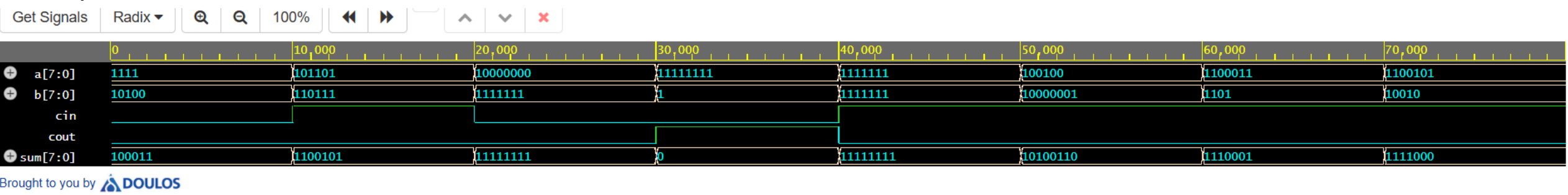
```
endmodule
```


SIMULATED OUTPUT WAVEFORMS OF 8 BIT RIPPLE CARRY ADDER

Decimal format -



Binary format -



Simulated Test Cases

- ▶ **Case 1 (0 – 10 ns):** $A = 15, B = 20, C_{\text{in}} \rightarrow 0$ {Sum = 35, $C_{\text{out}} = 0$ } .
Basic addition without input carry or output overflow.
- ▶ **Case 2 (10 – 20 ns):** $A = 45, B = 55, C_{\text{in}} \rightarrow 1$ {Sum = 101, $C_{\text{out}} = 0$ } .
Tests active carry-in inclusion ($45 + 55 + 1$).
- ▶ **Case 3 (20 – 30 ns):** $A = 128, B = 127, C_{\text{in}} \rightarrow 0$ {Sum = 255, $C_{\text{out}} = 0$ } .
Upper boundary 8-bit limit without overflow.
- ▶ **Case 4 (30 – 40 ns):** $A = 255, B = 1, C_{\text{in}} \rightarrow 0$ {Sum = 0, $C_{\text{out}} = 1$ } .
8-bit overflow scenario ($256 = 100000000$ (binary form)), triggering Carry-out High.
- ▶ **Case 5 (40 – 50 ns):** $A = 127, B = 127, C_{\text{in}} \rightarrow 1$ {Sum = 255, $C_{\text{out}} = 1$ } .
Maximum capacity test ($127 + 127 + 1 = 255$).

INPUT AND OUTPUT TABLE (FROM WAVEFORMS)

Time Interval (ps)	Input A (a[7:0])	Input B (b[7:0])	Carry In (Cin)	Sum (Sum[7:0])	Carry Out (Cout)	Verification (A+B+Cin)
0 – 10,000	15	20	0	35	0	$15 + 20 + 0 = 35$
10,000 – 20,000	45	55	1	101	0	$45 + 55 + 1 = 101$
20,000 – 30,000	128	127	0	255	0	$128 + 127 + 0 = 255$
30,000 – 40,000	255	1	0	0	1	$255 + 1 + 0 = 256(\text{Overflow})$
40,000 – 50,000	127	127	1	255	0	$127 + 127 + 1 = 255$
50,000 – 60,000	36	129	1	166	0	$36 + 129 + 1 = 166$
60,000 – 70,000	99	13	1	113	0	$99 + 13 + 1 = 113$
70,000 – 80,000	101	18	1	120	0	$101 + 18 + 1 = 120$

Waveform Output Analysis

 **Time Base:** Each test vector holds for **10,000 ps (10 ns)**.

 **Buses (A, B, Sum):** Multi-bit buses show decimal value boxes (e.g., 15, 20 → 35).

 **1-Bit Signals (C_{in}, C_{out}):**

- **Low (0):** Flat line at bottom baseline.
- **High (1):** Stepped-up top line.

 **Cursor Marker (40,000 ps):** Highlights A=127, B=127, C_{in}=1 resulting in Sum=255 and C_{out}=1.

Project Conclusion



Design Validation

The Verilog HDL description accurately implements 8-bit ripple carry arithmetic with clean modular full adder sub-modules.



100% Accuracy

All test vectors—including edge-case overflows and carry-in propagation—were verified successfully in EDA Playground.



Design Trade-off

While simple to implement, carry propagation delay grows linearly . Carry Lookahead Adders (CLA) serve as a faster alternative for high-speed systems.

References & Tools

-  **Digital Design: With an Introduction to the Verilog HDL** – M. Morris Mano & Michael D. Ciletti.
-  **EDA Playground Platform:** Interactive web tool for running simulation models using Icarus Verilog simulator.
-  **EPWave Waveform Viewer:** Open-source web-based VCD viewer used for waveform signal visualization.
-  **IEEE Std 1364-2005:** IEEE Standard for Verilog Hardware Description Language.

Thank You!

8-Bit Ripple Carry Adder | Verilog HDL Simulation Project

DONE BY K.ESHWAR

PROGRAMME:- VLSI

BATCH:- JUNE@26